

SQL

Christine Largouët

L'institut Agro Rennes Angers

Mai 2026

- SQL est un langage de programmation pour manipuler les bases de données
 - Définir le schéma de la base de données (**LDD**)
 - Charger les tables relationnelles (**LMD**)
 - Manipuler les données stockées (**LMD**)
 - Gérer la base de données (**LDD**) : sécurité, organisation physique

L'instruction **SELECT**

SELECT

attributs

expressions extraites

FROM expressions de tables

WHERE conditions de filtrage

GROUP BY conditions de groupe

HAVING conditions de filtrage sur les groupes

ORDER BY critères de tri

Les attributs

- Soit la table `client(identifiant, prenom, nom, ville)`

identifiant	prenom	nom	ville
1	Pierre	Dupond	Paris
2	Sabrina	Durand	Nantes
3	Julien	Martin	Lyon
4	David	Bernard	Marseille
5	Marie	Leroy	Grenoble

- Projection sur les attributs d'intérêt pour le résultat de la requête
- Les attributs choisis sont séparés par une virgule
- `SELECT nom FROM client;`
- `SELECT nom, prenom, ville FROM client;`

Les attributs

- Soit la table `client(identifiant, prenom, nom, ville)` suivante :

identifiant	prenom	nom	ville
1	Pierre	Dupond	Paris
2	Sabrina	Durand	Nantes
3	Julien	Martin	Lyon
4	David	Bernard	Marseille
5	Marie	Leroy	Grenoble

- `SELECT * FROM client;`
- `SELECT ville FROM client;`
- `SELECT ALL prenom, nom FROM client;`
- `SELECT DISTINCT prenom FROM client;`

Les alias

- Permet de renommer les tables et les colonnes pour faciliter l'écriture des requêtes:

identifiant	prenom	nom	ville
1	Pierre	Dupond	Paris
2	Sabrina	Durand	Nantes
3	Julien	Martin	Lyon
4	David	Bernard	Marseille
5	Marie	Leroy	Grenoble

- `SELECT ville AS v, FROM client;`
- `SELECT * FROM client AS c;`
- `SELECT * FROM client c;`

L'instruction **SELECT**

SELECT

attributs

expressions extraites

FROM expressions de tables

WHERE conditions de filtrage

GROUP BY conditions de groupe

HAVING conditions de filtrage sur les groupes

ORDER BY critères de tri

Conditions de filtrage

- Permet de sélectionner **les enregistrements** qui satisfont la condition indiquée dans la clause **WHERE**
- Utilisation des opérateurs de comparaison usuels. (=, <, >, >=, <=, <> ou !=)
- La comparaison porte un seul enregistrement (et non plusieurs)
- Autres opérateurs : **BETWEEN, IN, LIKE, IS NULL, IS NOT NULL**
- Opérateurs logiques : AND et OR

Conditions de filtrage

- Soit la table **produit(id, nom, categorie, stock, prix)**

id	nom	categorie	stock	prix
1	ordinateur	informatique	5	950
2	clavier	informatique	32	35
3	souris	informatique	16	30
4	crayon	fourniture	147	2

- **SELECT * FROM produit WHERE categorie = "informatique" AND stock < 20;**
(retourne : id 1 et id 3)
- **SELECT id, stock FROM produit WHERE nom = "ordinateur" OR nom = "clavier";**
(retourne : id 1 et id 2)

Conditions de filtrage : IN, BETWEEN

- Soit la table **produit(id, nom, categorie, stock, prix)**

id	nom	categorie	stock	prix
1	ordinateur	informatique	5	950
2	clavier	informatique	32	35
3	souris	informatique	16	30
4	crayon	fourniture	147	2

- **SELECT id FROM produit WHERE nom IN ("clavier", "souris");**
(retourne : id 2 et id 3)
- **SELECT nom FROM produit WHERE stock BETWEEN 20 AND 40;**
(retourne : id 2)

- **IN** : Permet de savoir si des valeurs appartiennent à une liste
 - Peut-être associé avec **NOT** (NOT IN)
- **BETWEEN** : L'intervalle peut être constitué de chaînes de caractères, de nombres ou de dates.
 - Attention l'inclusion ou non des bornes dépend des SGBD, consulter la documentation

Conditions de filtrage : LIKE

- Caractères joker pour les chaînes de caractères
 - "%" représente zéro, un ou plusieurs caractères
 - "_" représente un seul caractère
- Soit la table "**client**" suivante :

id	nom	ville
1	Léon	Lyon
2	Odette	Nice
3	Vivien	Nantes
4	Etienne	Lille

- **SELECT id, nom FROM client WHERE ville LIKE "N%";**
(id 2,3)

Conditions de filtrage : LIKE

- Usage des jokers
 - **LIKE** "%z" toutes les chaînes qui se terminent par **z**
 - **LIKE** "a%" toutes les chaînes qui commencent par **a**
 - **LIKE** "%f%" toutes les chaînes qui contiennent un **f**
 - **LIKE** "ma%tique" toutes les chaînes qui commencent par **ma** et finissent par **tique** comme *mathématique* ou *maieutique*
 - **LIKE** "a_c" toutes les chaînes de 3 caractères telles que "abc", "aac", "azc"

- Soit la table "**utilisateur**" suivante :

id	nom	date_inscription	fk_adresse_livraison_id	fk_adresse_facturation_id
23	Grégoire	2013-02-12	12	12
24	Sarah	2013-02-17	NULL	NULL
25	Anne	2013-02-21	13	14
26	Frédérique	2013-03-02	NULL	NULL

Utilisateurs sans adresse de livraison

- **SELECT * FROM utilisateur
WHERE fk_adresse_livraison_id IS NULL;**

Utilisateurs avec une adresse de livraison

- **SELECT * FROM utilisateur
WHERE fk_adresse_livraison_id IS NOT NULL;**

L'instruction **SELECT**

SELECT

attributs

expressions extraites

FROM expressions de tables

WHERE conditions de filtrage

GROUP BY conditions de groupe

HAVING conditions de filtrage sur les groupes

ORDER BY critères de tri

Critères de tri

- Définis à l'intérieur de la clause **ORDER BY**
 - Par défaut l'ordre d'affichage dépend de la base de données (généralement croissant)
 - "**ASC**" pour croissant
 - "**DESC**" pour décroissant
 - la valeur **NULL** est inférieure ou supérieure aux autres selon le SGBD
- **SELECT * FROM utilisateur ORDER BY nom;**

Utilisateurs retournés triés par ordre alphabétique ET pour ceux qui ont le même nom de famille, les trier par ordre décroissant d'inscription.

- **SELECT * FROM utilisateur ORDER BY nom,
date_inscription DESC**

Exercice

- Table **Plantes**(id, nom, hauteur_cm, type_sol, region)

id	nom	hauteur_cm	type_sol	region
1	Tomate	120	limoneux	Occitanie
2	Blé	85	argileux	Île-de-France
3	Tomate	110	sableux	Provence-Alpes
4	Tournesol	300	limoneux	Nouvelle-Aquitaine
5	Maïs	200	argileux	Bretagne
6	Blé	90	limoneux	Occitanie
7	Riz	150	limoneux	Provence-Alpes

- R1 : Lister les noms de plantes sans doublons
- R2 : Donner le nom et la hauteur des plantes cultivées sur un sol argileux
- R3 : Donner le nom et la région des plantes dont la hauteur est entre 100 et 200 cm.
- R4 : Trouver les plantes sur un sol limoneux dont la hauteur dépasse 100 cm.
- R5 : Afficher tous les enregistrements des régions autres que l'Occitanie et la Bretagne
- R6 : Donner les noms des plantes qui commencent par "To" classés par ordre alphabétique

Exercice - Correction

Plantes(id, nom, hauteur_cm, type_sol, region)

- R1 : Lister les noms de plantes sans doublons

```
SELECT DISTINCT nom FROM Plantes;
```

- R2 : Donner le nom et la hauteur des plantes cultivées sur un sol argileux

```
SELECT nom, hauteur_cm FROM Plantes WHERE  
type_sol = 'argileux';
```

- R3 : Donner le nom et la région des plantes dont la hauteur est entre 100 et 200 cm.

```
SELECT nom, region FROM Plantes WHERE hauteur_cm  
BETWEEN 100 AND 200;
```

Exercice - Correction

Plantes(id, nom, hauteur_cm, type_sol, region)

- R4 : Trouver les plantes sur un sol limoneux dont la hauteur dépasse 100 cm.

```
SELECT nom, hauteur_cm FROM Plantes WHERE type_sol =  
'limoneux' AND hauteur_cm > 100;
```

- R5 : Afficher tous les enregistrements des régions autres que l'Occitanie et la Bretagne

```
SELECT * FROM Plantes WHERE region NOT IN ('Occitanie',  
'Bretagne');
```

- R6 : Donner les noms des plantes qui commencent par "To" classés par ordre alphabétique

```
SELECT DISTINCT nom FROM Plantes WHERE nom LIKE "To%"  
ORDER BY nom;
```

PRODUIT CARTESIEN

- Construit **toutes les combinaisons possibles entre deux relations** (tables)
- $n * m$ lignes obtenues par le produit des deux tables de n et m lignes
- Opération à éviter

Vins1	Cru	Région	Couleur
	CHENAS	BEAUJOLAIS	ROUGE
	TOKAY	ALSACE	BLANC
	TAVEL	RHONE	ROSE

Années	Mill
	1980
	1985

Vins	Cru	Région	Couleur	Mill
	CHENAS	BEAUJOLAIS	ROUGE	1980
	TOKAY	ALSACE	BLANC	1980
	TAVEL	RHONE	ROSE	1980
	CHENAS	BEAUJOLAIS	ROUGE	1985
	TOKAY	ALSACE	BLANC	1985
	TAVEL	RHONE	ROSE	1985

• **SELECT * FROM vins1 , Années ;**

Jointure

VINS	Cru	Mill	Qualité
	VOLNAY	1983	A
	VOLNAY	1979	B
	CHABLIS	1983	A
	JULIENAS	1986	C

Jointure
sur l'attribut **Cru**

LOCALISATION	Cru	Région	QualMoy
	VOLNAY	BOURGOGNE	A
	CHABLIS	BOURGOGNE	A
	CHABLIS	CALIFORNIE	B

Cru = Cru

Résultat

VINSREG	Cru	Mill	Qualité	Région	QualMoy
	VOLNAY	1983	A	BOURGOGNE	A
	VOLNAY	1979	B	BOURGOGNE	A
	CHABLIS	1983	A	BOURGOGNE	A
	CHABLIS	1983	A	CALIFORNIE	B

JOINTURE NATURELLE

- Algèbre relationnelle : la jointure est une opération qui combine deux relations (tables) en fonction d'**une condition logique**, appelée condition de jointure.
 - Cette opération associe les tuples (lignes) de la première relation avec les tuples de la deuxième relation, mais uniquement lorsque la condition spécifiée est satisfaite.
 - Retourne les enregistrements lorsqu'il y a au moins une ligne dans chaque colonne qui correspond à la condition (attribut commun).
- Utilisation des mots clés (**INNER**) **JOIN** et **ON** qui spécifie la condition (l'égalité pour la majeure partie des cas)

Jointure Naturelle

Table 1 : Fermes

ID_Ferme	Nom_Ferme	Surface_ha	Région
1	Ferme_Soleil	50	Provence-Alpes
2	Ferme_Verde	75	Bourgogne
3	Ferme_Rivière	30	Grand-Est
4	Ferme_Montagne	60	Rhône-Alpes

Table 2 : Équipements

ID_Equipement	ID_Ferme	Type_Equipement	Année_Achat
101	1	Tracteur	2018
102	1	Moissonneuse	2020
103	2	Pulvérisateur	2017
104	3	Ensileuse	2019

- Jointure entre Fermes et Equipements

- Condition :**

ID_Ferme (table Fermes) = ID_Ferme (table Équipements)

- Résultat de la jointure

Résultat

ID_Ferme	Nom_Ferme	Surface_ha	Région	Type_Equipement	Année_Achat
1	Ferme_Soleil	50	Provence-Alpes	Tracteur	2018
1	Ferme_Soleil	50	Provence-Alpes	Moissonneuse	2020
2	Ferme_Verde	75	Bourgogne	Pulvérisateur	2017
3	Ferme_Rivière	30	Grand-Est	Ensileuse	2019

Jointure / Produit cartésien

Table 1 : Fermes

ID_Ferme	Nom_Ferme	Surface_ha	Région
1	Ferme_Soleil	50	Provence-Alpes
2	Ferme_Verde	75	Bourgogne
3	Ferme_Rivière	30	Grand-Est
4	Ferme_Montagne	60	Rhône-Alpes

Table 2 : Équipements

ID_Equipement	ID_Ferme	Type_Equipement	Année_Achat
101	1	Tracteur	2018
102	1	Moissonneuse	2020
103	2	Pulvérisateur	2017
104	3	Ensileuse	2019

- Jointure

ID_Ferme	Nom_Ferme	Surface_ha	Région	Type_Equipement	Année_Achat
1	Ferme_Soleil	50	Provence-Alpes	Tracteur	2018
1	Ferme_Soleil	50	Provence-Alpes	Moissonneuse	2020
2	Ferme_Verde	75	Bourgogne	Pulvérisateur	2017
3	Ferme_Rivière	30	Grand-Est	Ensileuse	2019

- Produit cartésien

Fermes.ID_Ferme	Fermes.Nom_Ferme	Surface_ha	Région	Équipements.ID_Equipement	Équipements.ID_Ferme	Type_Equipement	Année_Achat
1	Ferme_Soleil	50	Provence-Alpes	101	1	Tracteur	2018
1	Ferme_Soleil	50	Provence-Alpes	102	1	Moissonneuse	2020
1	Ferme_Soleil	50	Provence-Alpes	103	2	Pulvérisateur	2017
1	Ferme_Soleil	50	Provence-Alpes	104	3	Ensileuse	2019
2	Ferme_Verde	75	Bourgogne	101	1	Tracteur	2018
2	Ferme_Verde	75	Bourgogne	102	1	Moissonneuse	2020
2	Ferme_Verde	75	Bourgogne	103	2	Pulvérisateur	2017
2	Ferme_Verde	75	Bourgogne	104	3	Ensileuse	2019
3	Ferme_Rivière	30	Grand-Est	101	1	Tracteur	2018
3	Ferme_Rivière	30	Grand-Est	102	1	Moissonneuse	2020
3	Ferme_Rivière	30	Grand-Est	103	2	Pulvérisateur	2017
3	Ferme_Rivière	30	Grand-Est	104	3	Ensileuse	2019

16 enregistrements....

Jointure Naturelle

Table 1 : Fermes

ID_Ferme	Nom_Ferme	Surface_ha	Région
1	Ferme_Soleil	50	Provence-Alpes
2	Ferme_Verde	75	Bourgogne
3	Ferme_Rivière	30	Grand-Est
4	Ferme_Montagne	60	Rhône-Alpes

Table 2 : Équipements

ID_Equipement	ID_Ferme	Type_Equipement	Année_Achat
101	1	Tracteur	2018
102	1	Moissonneuse	2020
103	2	Pulvérisateur	2017
104	3	Ensileuse	2019

La requête SQL correspondant à la jointure de ces deux tables :

- **SELECT * FROM Fermes JOIN Équipements ON Fermes.ID_Ferme = Équipements.ID_Ferme;**
- **SELECT f.ID_Ferme, f.Nom_Ferme, e.Type_Equipement, e.Année_Achat
FROM Fermes f JOIN Équipements e
ON f.ID_Ferme = e.ID_Ferme;**

Exercice

- table "étudiant"

id_etudiant	prenom	nom
30	Joseph	Biblo
31	Paul	Bismuth
32	Jean	Michel
33	Ted	Bundy
34	Caroline	Martinez
35	Joséphine	Henry

- table "note"

id	id_examen	id_etudiant	matiere	note
788	45	30	Histoire-Geographie	10.5
789	87	33	Mathématiques	14
790	87	34	Mathématiques	4
791	45	31	Histoire-Geographie	15.5
792	45	32	Histoire-Geographie	8
793	87	31	Mathématiques	14
794	45	33	Histoire-Geographie	9.5
795	45	36	Histoire-Geographie	13
796	45	34	Histoire-Geographie	17
797	87	30	Mathématiques	7.5

- R1 - Donner la liste des matières par ordre alphabétique et sans doublon
- R2 - Donner le nom et prénom des étudiants ayant passé les examens et ayant eu plus de 10 à une matière

Exercice - Correction

- table "étudiant"

id_etudiant	prenom	nom
30	Joseph	Biblo
31	Paul	Bismuth
32	Jean	Michel
33	Ted	Bundy
34	Caroline	Martinez
35	Joséphine	Henry

- table "note"

id	id_examen	id_etudiant	matiere	note
788	45	30	Histoire-Geographie	10.5
789	87	33	Mathématiques	14
790	87	34	Mathématiques	4
791	45	31	Histoire-Geographie	15.5
792	45	32	Histoire-Geographie	8
793	87	31	Mathématiques	14
794	45	33	Histoire-Geographie	9.5
795	45	36	Histoire-Geographie	13
796	45	34	Histoire-Geographie	17
797	87	30	Mathématiques	7.5

- R1 - Donner la liste des matières par ordre alphabétique et sans doublon

```
SELECT DISTINCT matiere FROM note ORDER BY matiere ASC;
```

- R2 - Donner le nom et prénom des étudiants ayant passé les examens et ayant eu plus de 10 à une matière

```
SELECT DISTINCT nom, prenom  
FROM etudiant as e INNER JOIN note as n  
ON e.id_etudiant = n.id_etudiant  
WHERE note > 10;
```

Jointure et Condition

Table 1 : Fermes

ID_Ferme	Nom_Ferme	Surface_ha	Région
1	Ferme_Soleil	50	Provence-Alpes
2	Ferme_Verde	75	Bourgogne
3	Ferme_Rivière	30	Grand-Est
4	Ferme_Montagne	120	Rhône-Alpes

Table 2 : Quota_Subventions

ID_Quota	Min_Surface_ha	Max_Surface_ha	Subvention_ha
1	0	50	100
2	51	100	150
3	101	200	200

- Jointure entre Fermes et Quota_Subventions

- Condition :**

Min_Surface_ha $\leq$
Surface_ha $\leq$
Max_Surface_ha

- La jointure : Associer chaque ferme au barème de subvention correspondant à sa superficie, en utilisant une jointure avec une condition d'inégalité.

```
SELECT f.ID_Ferme, q.Subvention_ha
FROM Fermes f JOIN Quota_Subventions q
ON f.Surface_ha BETWEEN q.Min_Surface_ha AND q.Max_Surface_ha;
```

Jointure et Condition

Table 1 : Fermes

ID_Ferme	Nom_Ferme	Surface_ha	Région
1	Ferme_Soleil	50	Provence-Alpes
2	Ferme_Verde	75	Bourgogne
3	Ferme_Rivière	30	Grand-Est
4	Ferme_Montagne	120	Rhône-Alpes

Table 2 : Quota_Subventions

ID_Quota	Min_Surface_ha	Max_Surface_ha	Subvention_ha
1	0	50	100
2	51	100	150
3	101	200	200

- Jointure entre Fermes et Quota_Subventions

- Condition :**

Min_Surface_ha $\leq$
Surface_ha $\leq$
Max_Surface_ha

- Résultat de la jointure

Résultat

ID_Ferme	Nom_Ferme	Surface_ha	Subvention_ha
1	Ferme_Soleil	50	100
2	Ferme_Verde	75	150
3	Ferme_Rivière	30	100
4	Ferme_Montagne	120	200

Auto-jointure

- Une auto-jointure n'est qu'un cas particulier de jointure.
- Au lieu de joindre deux relations différentes, on réalise une jointure entre deux copies de la même relation.

Table : Produits

ID_Produit	Nom_Produit	Prix_Unitaire	Catégorie
1	Engrais_Bio	150	Intrants
2	Engrais_Chimique	100	Intrants
3	Pesticide_Naturel	180	Intrants
4	Herbicide	130	Intrants
5	Fertilisant_Luxe	250	Intrants

- Trouver tous les produits plus chers qu'un produit donné, par exemple **Herbicide**.

```
SELECT P1.Nom_Produit AS Produit_Cible, P2.Nom_Produit AS
Produit_Plus_Cher, P2.Prix_Unitaire
FROM Produits P1 JOIN Produits P2
ON P2.Prix_Unitaire > P1.Prix_Unitaire
WHERE P1.Nom_Produit = "Herbicide";
```

- Dans la jointure interne (**INNER JOIN**) les tuples qui ne peuvent pas être joints sont éliminés du résultat
- Dans la jointure externe (**OUTER JOIN**) les tuples qui ne peuvent pas être joints sont conservés dans le résultat et les attributs qui ne peuvent pas être remplis sont remplacés par la valeur **NULL**
- Dans la jointure externe il est possible de ne conserver que la partie gauche ou la partie droite de la jointure à l'aide des opérateurs : **LEFT JOIN (LEFT OUTER JOIN)** , **RIGHT JOIN (RIGHT OUTER JOIN)**

JOINTURE

chauffeur

Pseudo	Prénom	Immat
Fanny35	Fanny	BX-201-pl
PoloPol	Paul	AY-471-IT
M-35	Marie	TC-965-IJ

vehicule

Immat	Type	KM
BX-201-pl	C5	25124
AY-471-IT	Mégane	45120
TR-876-OI	206	284500
TC-965-IJ	Kangoo	170000

Résultat
de la jointure
naturelle

Pseudo	Prenom	Immat	Type	KM
Fanny35	Fanny	BX-201-pl	C5	25124
Polopo	Paul	AY-471-IT	Mégane	45120
M-35	Marie	TC-965-IJ	Kangoo	170000

JOINTURE EXTERNE

chauffeur

Pseudo	Prénom	Immat
Fanny35	Fanny	BX-201-pl
PoloPol	Paul	AY-471-IT
M-35	Marie	TC-965-IJ

vehicule

Immat	Type	KM
BX-201-pl	C5	25124
AY-471-IT	Mégane	45120
TR-876-OI	206	284500
TC-965-IJ	Kangoo	170000

La requête SQL correspondant à la jointure externe de ces deux tables :

- SELECT Pseudo, Immat, Type, KM**

FROM chauffeur as c RIGHT OUTER JOIN vehicule as v

ON c.Immat = v.Immat;

RIGHT JOIN retourne toutes les lignes de la colonne de droite.

JOINTURE EXTERNE

chauffeur

Pseudo	Prénom	Immat
Fanny35	Fanny	BX-201-pl
PoloPol	Paul	AY-471-IT
M-35	Marie	TC-965-IJ

vehicule

Immat	Type	KM
BX-201-pl	C5	25124
AY-471-IT	Mégane	45120
TR-876-OI	206	284500
TC-965-IJ	Kangoo	170000

Résultat
de la jointure
externe droite

Pseudo	Immat	Type	KM
Fanny35	BX-201-pl	C5	25124
Polopo	AY-471-IT	Mégane	45120
NULL	TR-876-OI	206	284500
M-35	TC-965-IJ	Kangoo	170000

chauffeur

Pseudo	Prénom	Immat
Fanny35	Fanny	BX-201-pl
PoloPol	Paul	AY-471-IT
M-35	Marie	TC-965-IJ

vehicule

Immat	Type	KM
BX-201-pl	C5	25124
AY-471-IT	Mégane	45120
TR-876-OI	206	284500
TC-965-IJ	Kangoo	170000

Résultat
de la jointure
externe droite

Pseudo	Immat	Type	KM
Fanny35	BX-201-pl	C5	25124
Polopo	AY-471-IT	Mégane	45120
NULL	TR-876-OI	206	284500
M-35	TC-965-IJ	Kangoo	170000

Permet de répondre à la requête : "Quelles sont les voitures qui n'ont pas de chauffeur ?"

- **SELECT Immat**
FROM chauffeur as c RIGHT JOIN vehicule as v
ON c.Immat = v.Immat
WHERE Pseudo = NULL;

Exemple : JOINTURE EXTERNE

Table 1 : Fermes

ID_Ferme	Nom_Ferme	Surface_ha	Région
1	Ferme_Soleil	50	Provence-Alpes
2	Ferme_Verde	75	Bourgogne
3	Ferme_Rivière	30	Grand-Est
4	Ferme_Montagne	60	Rhône-Alpes

Table 2 : Équipements

ID_Equipement	ID_Ferme	Type_Equipement	Année_Achat
101	1	Tracteur	2018
102	1	Moissonneuse	2020
103	2	Pulvérisateur	2017
104	3	Ensileuse	2019
105	NULL	Semoir	2021

- On souhaite afficher :
 - Toutes les fermes, même celles sans équipements (jointure externe gauche).
 - Toutes les équipements, même ceux non attribués à une ferme (jointure externe droite).
 - Une vue combinée de ces données (jointure externe complète).

Exemple : JOINTURE EXTERNE

- Toutes les fermes, même celles sans équipements: Jointure externe gauche (**LEFT JOIN**)

```
SELECT Fermes.ID_Ferme, Fermes.Nom_Ferme,  
Fermes.Surface_ha, Équipements.Type_Equipement,  
Équipements.Année_Achat  
FROM Fermes LEFT JOIN Équipements  
ON Fermes.ID_Ferme = Équipements.ID_Ferme;
```

Résultat

ID_Ferme	Nom_Ferme	Surface_ha	Type_Equipement	Année_Achat
1	Ferme_Soleil	50	Tracteur	2018
1	Ferme_Soleil	50	Moissonneuse	2020
2	Ferme_Verde	75	Pulvérisateur	2017
3	Ferme_Rivière	30	Ensileuse	2019
4	Ferme_Montagne	60	NULL	NULL

Exemple : JOINTURE EXTERNE

- Jointure externe droite (**RIGHT JOIN**)

```
SELECT Fermes.ID_Ferme, Fermes.Nom_Ferme,  
Fermes.Surface_ha, Équipements.Type_Equipement,  
Équipements.Année_Achat  
FROM Fermes RIGHT JOIN Équipements  
ON Fermes.ID_Ferme = Équipements.ID_Ferme;
```

Résultat

ID_Ferme	Nom_Ferme	Surface_ha	Type_Equipement	Année_Achat
1	Ferme_Soleil	50	Tracteur	2018
1	Ferme_Soleil	50	Moissonneuse	2020
2	Ferme_Verde	75	Pulvérisateur	2017
3	Ferme_Rivière	30	Ensileuse	2019
NULL	NULL	NULL	Semoir	2021

Exemple : JOINTURE EXTERNE

- Jointure Externe Complète (**FULL OUTER JOIN**)

```
SELECT Fermes.ID_Ferme, Fermes.Nom_Ferme,  
Fermes.Surface_ha, Équipements.Type_Equipement,  
Équipements.Année_Achat  
FROM Fermes FULL OUTER JOIN Équipements  
ON Fermes.ID_Ferme = Équipements.ID_Ferme;
```

Résultat

ID_Ferme	Nom_Ferme	Surface_ha	Type_Equipement	Année_Achat
1	Ferme_Soleil	50	Tracteur	2018
1	Ferme_Soleil	50	Moissonneuse	2020
2	Ferme_Verde	75	Pulvérisateur	2017
3	Ferme_Rivière	30	Ensileuse	2019
4	Ferme_Montagne	60	NULL	NULL
NULL	NULL	NULL	Semoir	2021

L'instruction **SELECT**

SELECT

attributs

expressions extraites

FROM expressions de tables

WHERE conditions de filtrage

GROUP BY conditions de groupe

HAVING conditions de filtrage sur les groupes

ORDER BY critères de tri

Pourquoi des groupes ?

- Permet de répondre à des requêtes qui concernent plusieurs lignes :
 - Quel est le nombre de voitures d'un chauffeur ?
 - Quelle est la somme des achats effectués dans une même commande ?
 - Quel est le prix moyen d'un produit par couleur?
- Utilisation possible des agrégats pouvant être appliqués sur les attributs : **MIN()** , **MAX()** , **AVG()** , **SUM()** , **COUNT()**

GROUP BY

- Donner leur nom et le nombre de voyages effectués par chaque chauffeur

```
SELECT NomChauffeur, COUNT(NumVoyage)  
FROM VOYAGE JOIN CHAUFFEUR  
ON  
VOYAGE.NumChauffeur=CHAUFFEUR.NumChauffeur  
GROUP BY NumChauffeur;
```

GROUP BY

Table : Produits_Phyto

ID	Produit	Prix	Date_Achat	Client
1	Herbicide_Agrox	45.00	2023-03-15	Client_A
2	Fongicide_Fungimax	60.50	2023-06-20	Client_B
3	Insecticide_BugAway	30.75	2023-07-10	Client_A
4	Regulateur_Croissance	25.00	2023-09-05	Client_C
5	Desherbant_Killer	75.00	2023-10-01	Client_B
6	Engrais_BioGrowth	40.00	2023-11-22	Client_A

- Quelle est la somme des achats effectués pour chaque client ?

```
SELECT Client, SUM(Prix)  
FROM Produits_Phyto  
GROUP BY Client;
```

- Quelle le plus cher de chaque client ?

```
SELECT Client, MAX(Prix)  
FROM Produits_Phyto  
GROUP BY Client;
```

HAVING

- **HAVING** permet de porter une condition sur les groupes (ou toute la table si pas de groupes)
- Attention à ne pas confondre le **WHERE** et le **HAVING**
 - Le **WHERE** porte sur les enregistrements un à un
 - Le **HAVING** porte sur le groupe
- Il est possible de combiner un **WHERE** puis un **HAVING**

HAVING

- Donner le nombre de voyages effectués par les chauffeurs ayant effectué plus de 100 trajets

```
SELECT NomChauffeur, COUNT(NumVoyage)  
FROM VOYAGE JOIN CHAUFFEUR  
ON VOYAGE.NumChauffeur=CHAUFFEUR.NumChauffeur  
GROUP BY NumChauffeur  
HAVING COUNT(NumVoyage) > 100;
```

HAVING

Table : Produits_Phyto

ID	Produit	Prix	Date_Achat	Client
1	Herbicide_Agrox	45.00	2023-03-15	Client_A
2	Fongicide_Fungimax	60.50	2023-06-20	Client_B
3	Insecticide_BugAway	30.75	2023-07-10	Client_A
4	Regulateur_Croissance	25.00	2023-09-05	Client_C
5	Desherbant_Killer	75.00	2023-10-01	Client_B
6	Engrais_BioGrowth	40.00	2023-11-22	Client_A

- Quelle est la liste des clients qui ont commandé plus de 80€, toutes commandes confondues ?

```
SELECT client, SUM(Prix)
FROM Produits_Phyto
GROUP BY Client
HAVING SUM(Prix) > 80
```

Exercice

- table "**Etudiant**"

id_etudiant	prenom	nom
30	Joseph	Biblo
31	Paul	Bismuth
32	Jean	Michel
33	Ted	Bundy
34	Caroline	Martinez
35	Joséphine	Henry

- table "**Note**"

id	id_examen	id_etudiant	matiere	note
788	45	30	Histoire-Geographie	10.5
789	87	33	Mathématiques	14
790	87	34	Mathématiques	4
791	45	31	Histoire-Geographie	15.5
792	45	32	Histoire-Geographie	8
793	87	31	Mathématiques	14
794	45	33	Histoire-Geographie	9.5
795	45	36	Histoire-Geographie	13
796	45	34	Histoire-Geographie	17
797	87	30	Mathématiques	7.5

- R1 - Donner le nom des élèves qui n'ont pas passé d'examen
- R2 - Donner la moyenne des notes par matière
- R3 - Donner les noms des élèves qui ont eu une moyenne de plus de 10

Exercice - Correction

- R1 - Donner le nom des élèves qui n'ont pas passé d'examen

```
SELECT DISTINCT e.nom  
FROM Etudiant e LEFT OUTER JOIN Note n  
ON e.id_etudiant = n.id_etudiant  
WHERE n.id_examen = NULL;
```

- R2 - Donner la moyenne des notes par matière

```
SELECT matiere, AVG(note)  
FROM Note  
GROUP BY matiere;
```

- R3 - Donner les noms des élèves qui ont eu une moyenne de plus de 10

```
SELECT nom  
FROM Etudiant e JOIN Note n  
ON e.id_etudiant = n.id_etudiant  
GROUP BY n.id_etudiant  
HAVING AVG(note) > 10
```


Imbrication de SELECT

- Les requêtes peuvent être combinées de différentes manières :
 - **composition** : on peut utiliser une requête SQL à laquelle on a attribué un nom dans une autre requête
 - **imbrication** : à l'intérieur d'une requête SQL peut exister une autre requête SQL à l'intérieur de la clause **WHERE**

Imbrication de SELECT

- Soit la table **produit**(numproduit, nom, libelle, poids)
- Quelle est la liste des produits et leur poids de poids supérieur au poids de l'article numéro 17 ?

```
SELECT p.nom, p.poids
FROM produit p
WHERE p.poids >= (SELECT p.poids FROM
produit p WHERE p.numproduit = 17);
```

Imbrication de SELECT

- La requête interne peut retourner plusieurs valeurs
- Possibilité d'utiliser les opérateurs suivants :
 - **IN** teste l'appartenance au résultat du **select** interne
 - **ANY** combinée avec un opérateur de comparaison. Par exemple **>** teste si l'expression est supérieure à au moins un résultat de la requête interne
 - **ALL** idem avec tous
 - **EXISTS (NOT EXISTS)** retourne VRAI si la requête interne retourne un résultat non vide (vide)

Imbrication de SELECT

- La liste des produits qui figurent au moins une fois dans une commande en quantité supérieure à 10

```
SELECT p.nom  
FROM produit p  
WHERE p.numproduit IN (SELECT  
c.refproduit FROM Commande c  
WHERE c.quantite >= 10)
```

- Requête également possible avec une jointure (et moins coûteuse)

Conclusion

- L'instruction SELECT a un très grand pouvoir d'expression
- Repose sur l'algèbre relationnelle qui permet des opérations élégantes et peu coûteuses (jointure naturelle, jointure externe, opérateurs ensemblistes)
- On peut combiner et imbriquer des requêtes
- La syntaxe peut légèrement varier d'un SGBD à l'autre mais SQL est le langage le plus répandu pour définir des requêtes.

- Langage de définition des données
- Définition des schémas logiques
 - Relations (structure, données)
 - Clés
- Définition des types de données et des contraintes
- Commande **CREATE**, **ALTER** et **DROP**

CREATION DE TABLES

- Soit une relation :

Relation1 (Att1:Dom1, . . . , Att2:Dom2)

- La définition d'une table suit la syntaxe suivante :

```
CREATE TABLE Relation1 (  
Att1 Dom1,  
.  
.  
.  
Att2 Dom2) ;
```


Domaines, Types

- Les types diffèrent selon les SGBD employés (ici MySQL)
- Chaînes de caractères
 - **CHAR(M)** : Chaîne de longueur fixe (ex. : "AB123").
 - **VARCHAR(M)** : Chaîne de longueur variable (ex. : "Description produit").
 - **TEXT** : Texte long, jusqu'à 64 Ko.
- Numériques
 - **INT / INTEGER** : Entier (ex. : 42)
 - **TINYINT / SMALLINT / BIGINT** : Entiers de tailles différentes.
 - **DECIMAL(M, D)** : Nombre décimal exact (ex. : 123.45)
 - **FLOAT / DOUBLE** : Nombre décimal
- Date et Heure
 - **DATE** : Date (YYYY-MM-DD, ex. : "2025-01-13").
 - **DATETIME** : Date et heure (YYYY-MM-DD HH:MM:SS).
 - **TIMESTAMP** : Date et heure avec fuseau horaire.
 - **TIME** : Heure (HH:MM:SS).
 - **YEAR** : Année (ex. : 2025).

Exemple - Création de table

```
CREATE TABLE Parcelles (  
    ID_Parcelle INT, -- Identifiant unique pour chaque parcelle  
    Nom_Parcelle VARCHAR(100), -- Nom de la parcelle("Parcelle_A")  
    Surface DECIMAL(10, 2), -- Surface en hectares (ex. : 12.50)  
    Culture VARCHAR(50), -- Type de culture (ex. : "Blé", "Maïs")  
    Date_Semence DATE, -- Date de semence  
    Date_Recolte DATE, -- Date prévue de récolte  
    Commentaires TEXT -- Notes ou remarques sur la parcelle  
);
```

Contraintes

- Permettent de définir plus précisément le contenu des colonnes
- **NOT NULL** : Contenu du champ obligatoire
- **UNIQUE** : Unicité des valeurs de l'attribut
- **AUTO_INCREMENT** : Incrémentation automatique du nombre
- **PRIMARY KEY** : Clé primaire
- **FOREIGN KEY** : Clé étrangère
- Certaines contraintes en impliquent d'autres :
 - PRIMARY KEY $\Rightarrow$ NOT NULL
 - PRIMARY KEY $\Rightarrow$ UNIQUE

Exemple - Création de table

```
CREATE TABLE Exploitants (  
    ID_Exploitant INT AUTO_INCREMENT PRIMARY KEY,  
    Nom_Exploitant VARCHAR(100) NOT NULL,  
    Adresse VARCHAR(255),  
    Telephone VARCHAR(15)  
);  
  
CREATE TABLE Parcelles (  
    ID_Parcelle INT AUTO_INCREMENT PRIMARY KEY,  
    Nom_Parcelle VARCHAR(100) NOT NULL,  
    Surface DECIMAL(10, 2) NOT NULL,  
    Culture VARCHAR(50) NOT NULL,  
    Date_Semence DATE,  
    Date_Recolte DATE,  
    Commentaires TEXT,  
    ID_Exploitant INT,  
    FOREIGN KEY (ID_Exploitant) REFERENCES  
Exploitants(ID_Exploitant)  
);
```

- Permet de modifier les données et les tables des bases de données relationnelles
- Commandes INSERT, UPDATE et DELETE
- Commande **INSERT INTO**

```
INSERT INTO nom_table VALUES (val1, ..., valn) ;
```

```
INSERT INTO nom_table(att1, ..., attn)  
    VALUES (val1, ..., vali, ..., valn) ;
```

```
INSERT INTO nom_table(att1, ..., attb) SELECT ...  
FROM ... ;
```

Exemple - INSERTION

- Insertion explicite en spécifiant les colonnes

```
INSERT INTO Exploitants (Nom_Exploitant, Adresse, Telephone)  
VALUES  
("Paul Durand", "10 rue des Vignes, 34000 Montpellier",  
"0456789102"),  
("Sophie Legrand", "20 rue des saules, 31000 Toulouse", NULL);
```

- Insertion implicite sans spécifier les colonnes
DEFAULT permet à MySQL de générer automatiquement des valeurs par défaut (ID auto-incrémenté)

```
INSERT INTO Exploitants  
VALUES  
(DEFAULT, "Luc Moreau", "25 rue des Champs, 59000 Lille",  
"0623456789"),  
(DEFAULT, "Anne Leroy", "5 impasse des Vergers, 44000 Nantes",  
"0321456789");
```

Mise à jour des données

- Commande **UPDATE**
- Permet de modifier les valeurs d'une ou plusieurs colonnes dans un ou plusieurs enregistrements d'une table.
- Toujours utilisée avec une condition **WHERE**

```
UPDATE nom_table  
SET att1 = val1, att2 = val2, ...  
WHERE condition;
```

Exemple - Mise à jour des données

```
UPDATE Exploitants
```

```
SET Adresse = "20 rue des Arts, Lyon"
```

```
WHERE Nom_Exploitant = "Jean Martin";
```

Modifier le téléphone de tous les 07 en rajoutant +33

```
UPDATE Exploitants
```

```
SET Telephone = CONCAT(' +33 ', Telephone)
```

```
WHERE Telephone LIKE '07%';
```


Création de vues

- Simplifie les requêtes en les enregistrant sous forme d'objet dans la base de données (table virtuelle)
- Permet de représenter des données réparties dans plusieurs tables
- Simplification des requêtes complexes et limite le risque d'erreur
- Permet de gérer les droits associés aux données (données sensibles)

```
CREATE VIEW nom_vue [att1, ...] AS SELECT ...;
```

- Suppression de la vue

```
DROP VIEW nom_vue
```

- Les vues ne stockent pas les données, mais la requête.
- Les modifications des tables sous-jacentes sont répercutées dans la vue.

Exemple - Création de vues

```
CREATE VIEW MaVue AS  
SELECT  p.ID_Parcelle, p.Surface_ha,  
p.Type_Culture, p.Nom_Exploitant  
FROM  Parcelles p JOIN Exploitants e  
ON  p.ID_Exploitant = e.ID_Exploitant;
```

- Utilisation de la vue

```
SELECT *  
FROM MaVue  
WHERE Surface_ha>12;
```

- Suppression de la vue

```
DROP VIEW MaVue
```

Suppression

- La suppression de la vue ne supprime pas les tables associées
- Pour supprimer une table et les données qu'elle contient (opération irréversible)

DROP [IF EXISTS] nom_table

- L'option **CASCADE** supprime tous les objets dépendants de la table (vues, des contraintes de clé étrangère).

DROP [IF EXISTS] nom_table CASCADE

- **ON DELETE CASCADE** - aussi possible dans la création de la table avec la clé étrangère

Création d'index

- But : amélioration de la performance, accélération de l'exécution des requêtes (transparent pour les requêtes)
- Fonctionne comme un répertoire pour référencer des lignes, optimise les jointures et les tris
- Par défaut : clé primaire
- Occupe de l'espace disque (ralentit INSERT, UPDATE et DELETE)

```
CREATE INDEX nom_index  
ON nom_table(colonne) ;
```

- Bien choisir les colonnes à indexer : priorité aux colonnes utilisées dans les filtres ou les jointures.
- Ne pas abuser des index : trop d'index peut nuire aux performances lors de l'écriture.

Exemple - Index

- Accélérer la recherche sur le type de culture

```
CREATE INDEX idx_type_culture  
ON Cultures (Type_Culture) ;
```

- Usage de cet index dans la requête

```
SELECT *  
FROM Cultures  
WHERE Type_Culture = 'Blé' ;
```

- Empêcher des doublons sur l'association entre la culture et la date de plantation :

```
CREATE UNIQUE INDEX idx_unique_culture_date  
ON Cultures (Type_Culture, Date_Plantation) ;
```

- Index multi-colonnes pour optimiser les requêtes filtrant par type de culture et client

```
CREATE INDEX idx_culture_client  
ON Cultures (Type_Culture, ID_Client) ;
```

CONCLUSION

- Conception de la base de données : **essentiel pour garantir la cohérence des données**
 - Nécessité de comprendre le système et le besoin
 - Définition des tables (3FN)
 - Lien entre elles via les clés primaires et clés étrangères
- **SQL** : Langage de référence incontournable des bases de données
 - Manipulation des données avec des instructions simples et puissantes (**SELECT**)
 - Optimisation pour des requêtes plus efficaces (vues, index)
- Les bases de données jouent un rôle crucial dans les systèmes d'Intelligence Artificielle (IA)
- Elles sont utilisées pour stocker, organiser et interroger de grandes quantités de données nécessaires à l'entraînement des modèles d'IA.